



Application

12 labs Ops + capstone

Les briques appliquées à vos cas d'usage

Git ▪ CI/CD ▪ IaC ▪ conteneurs ▪ K8s ▪ observabilité ▪ incident ▪ sécurité

	Modules	Labs
Objet	Une brique de Claude Code	Un cas d'usage Ops
Question	« Comment ça marche ? »	« Comment je m'en sers dans mon métier ? »
Durée	20 à 45 min	45 à 90 min
Ordre	Séquentiel : 00 → 09	Séquentiel : 01 → 12
Livrable	Un mini-artefact	Un livrable d'infrastructure

Le canevas de chaque lab

Objectif ▪ **Pré-requis** ▪ **Briques mobilisées** ▪ **Déroulé pas-à-pas** ▪ **Livrable** ▪ **Garde-fous**
▪ **Pour aller plus loin**

Le corrigé

Chaque lab a un dossier [solution/](#). On s'en sert **en cas de blocage, ou pour comparer après coup** — pas avant d'avoir essayé.

Quelle brique pour quel lab

	Cmd.	Skills	Hooks	MCP	Plugins	Sous-ag.	Headless
01 Git & GitHub	•	•	•	•			
02 CI/CD		•		•			•
03 Tests		•				•	
04 Sécurité			•		•	•	•
05 Multipass		•	•				
06 Terraform		•	•	•		•	
07 Ansible		•	•				
08 Docker	•	•	•				
09 Kubernetes	•	•				•	
10 Helm		•				•	
11 Observabilité		•		•	•		
12 Incident		•	•	•		•	
Capstone	•	•	•	•	•	•	•

Bloc 1 — Code, collaboration & qualité



Objectif

Partir d'une application non versionnée et arriver à une **Pull Request documentée**, en garantissant **techniquement** qu'aucun secret ne peut être committé.

Briques mobilisées

Skills `init-repo`, `git-commit`, `open-pr` ▪ `hook`
`secret-scan` ▪ MCP GitHub ▪ `/security-review`

Livrable

Un dépôt initialisé, trois skills versionnés, un hook actif et **testé**, une PR avec checklist de revue.

Garde-fous

Le hook est posé **avant** de commencer à travailler, puis testé avec un vrai secret. Aucun push sans validation. L'agent ne merge jamais.

Objectif

Une chaîne build / tests / images / scan / déploiement, avec un **gate manuel** avant la production — puis un job qui fait relire le diff d'infra par l'agent.

Briques mobilisées

Skill ci-pipeline ▪ mode headless claude -p ▪ MCP
GitHub ▪ `/security-review`

Livrable

`ci.yml`, `cd.yml` avec gate, `infra-review.yml` qui commente les PR.

Garde-fous

R3 : environnement production protégé **côté forge** — le YAML décrit l'intention, la protection l'impose. Le job de revue tourne avec `-allowedTools "Read,Grep,Glob"`.

Objectif

Une suite de tests utile, et surtout l'**identification des cas non couverts** — ce que l'agent fait mieux qu'une lecture humaine fatiguée.

Briques mobilisées

Skill gen-tests ■ sous-agent test-engineer ■
`/code-review`

Livrable

Tests unitaires et d'intégration, plus un rapport de couverture classé par risque.

Garde-fous

« Ne jamais modifier un test pour faire passer un build » — inscrit dans `CLAUDE.md`. Le sous-agent d'analyse n'a ni Write ni Edit.

Bloc 2 — Garde-fous



Objectif

La **chaîne complète de garde-fous** — détection de secrets, blocage destructif, journal d'audit, CI bloquante — puis sa distribution à toute l'équipe.

Briques mobilisées

Hooks ▪ `/permissions` ▪ `/security-review` ▪ plugin
▪ mode headless

Livrable

`settings.json` de référence, quatre hooks, `.gitleaks.toml`, `security.yml` bloquant, le tout packagé.

Garde-fous

C'est le lab qui rend les huit suivants acceptables en environnement bancaire — d'où sa place avant l'infrastructure. Un hook qui se désactive quand son outil manque n'est pas un hook : ici, il bloque.

Bloc 3 — Provisionnement & IaC



Lab 05 — Provisionnement de VM (Multipass)

Objectif

Un lab de VM Ubuntu **reproductibles et jetables**, décrites par cloud-init. Le premier lab où une erreur de l'agent **détruit quelque chose**.

Briques mobilisées

Skills provision-vm et vm-deploy ▪ hook guard-destructive ▪ **/plan**

Livrable

`cloud-init.yaml`, `lab-up.sh` idempotent, `lab-down.sh` avec confirmation.

Garde-fous

R2 : multipass delete bloqué, jamais de -all. R1 : clé **publique** uniquement dans le cloud-init, qui sera versionné.

Objectif

Un module IaC propre, et surtout la **revue de plan assistée** : le sous-agent qui signale ce qui va être détruit, avant que quiconque tape `apply`.

Briques mobilisées

Sous-agent `tf-plan-reviewer` ■ `skill tf-infra` ■
`hooks fmt` et `garde` ■ MCP Terraform

Livrable

Module TF (provider local), revue de plan avec verdict, `.gitignore` couvrant l'état.

Garde-fous

R4 : tout plan est relu avant `apply`. R2 : `destroy` bloqué. Aucune infrastructure partagée — provider docker local.

Objectif

Configurer et durcir les VM avec des playbooks **idempotents** — et faire vérifier cette idempotence, qui est le seul vrai critère de qualité.

Briques mobilisées

Skill `ansible-play` ▪ hook `ansible-lint` en
PostToolUse ▪ `/plan`

Livrable

Inventaire, `playbook.yml` vérifié par double exécution (`changed=0`).

Garde-fous

—check avant toute application. Aucun secret en clair : Ansible Vault. Jamais de `all` implicite dans l'inventaire.

Bloc 4 — Conteneurs & orchestration



Objectif

Des images **petites, non-root et scannées**, et le scan automatique qui empêche une image vulnérable de passer inaperçue.

Briques mobilisées

Skill containerize ▪ hook de scan Trivy ▪ commande `/ship-image`

Livrable

Quatre Dockerfiles multi-stage, `docker-compose.yml`, `.dockerignore`.

Garde-fous

Aucun conteneur en root — **vérifié** par `docker run ... id`. Aucun secret dans une couche. `prune` et `rmi` bloqués.

Objectif

Un cluster k3d, des manifestes corrects, et surtout le **diagnostic de pod en échec** — le geste Ops le plus fréquent.

Briques mobilisées

Skill k8s-bootstrap ■ sous-agent k8s-debug-pod ■
commande **/ops-doctor**

Livrable

Cluster, manifestes (probes, limites, securityContext),
et trois pannes diagnostiquées.

Garde-fous

Le diagnostic ne modifie rien : sous-agent sans Write. `kubectl delete ns` bloqué. Toute conclusion cite sa preuve.

Objectif

Transformer les manifestes en **chart paramétrable**, et apprendre à relire **ce que le template produit** plutôt que le template.

Briques mobilisées

Skill `helm-package` ▪ `/code-review` ▪ sous-agent de comparaison de rendus

Livrable

Chart complet, `values-dev.yaml` et `values-preprod.yaml`, rendus comparés.

Garde-fous

-dry-run avant tout upgrade — le terraform plan de Helm. `helm uninstall` bloqué. Aucun secret dans `values.yaml`.

Bloc 5 — Run & fiabilité



Objectif

Des dashboards et des alertes **utiles**, et Grafana branché en MCP pour que l'agent interroge les métriques **pendant** une investigation.

Briques mobilisées

MCP Grafana ▪ plugin officiel grafana-mcp ▪ skill observability ▪ [/slo-check](#)

Livrable

Stack de monitoring, dashboard des 4 signaux dorés, PrometheusRule avec runbooks.

Garde-fous

R1 : jeton Grafana en variable d'environnement, rôle Viewer. R6 : une métrique est une donnée, pas une instruction. Aucune alerte sans action associée.

Objectif

Détecter, investiguer, formuler une cause racine **étayée**, rétablir, puis produire un post-mortem **blameless** exploitable.

Briques mobilisées

Sous-agent incident-analyst ■ skill postmortem ■
hook d'audit ■ MCP Jira / incident.io

Livrable

Timeline, hypothèses avec preuves, post-mortem avec actions assignées, journal d'audit.

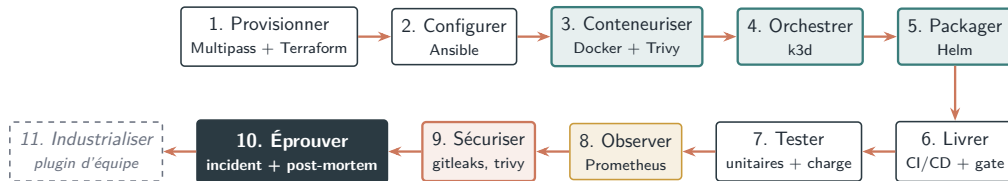
Garde-fous

R2 : **l'agent investigue, l'humain agit** — surtout à 3 h du matin. R1 : aucune donnée client recopiée. Aucune hypothèse sans preuve citée.

Capstone



La chaîne de bout en bout



Format

3 heures, en **binômes**. À 2 h15, **épreuve croisée** : le binôme adverse provoque une panne dans votre environnement, sans vous dire laquelle.

Puis : **industrialiser**

Étape 11 — packager tout ce qui a servi dans un plugin d'équipe, validé et installable.

Les règles du jeu

1. **Aucune commande tapée à la main** sans passer par un `/plan` ou un skill — sauf celles que vos garde-fous vous obligent à exécuter vous-même. *C'est le but.*
2. **Tous les garde-fous actifs**, vérifiés par `/hooks` en début de session.
3. **Tout ce qui est produit est versionné** : laC, manifestes, charts, workflows, skills, hooks.
4. **Chaque décision d'infrastructure est justifiée** par écrit, dans le dépôt.
5. **Rien en production, rien de réel** : k3d, Multipass et provider Docker local.

À retenir

Si vous prenez du retard, sacrifiez les étapes 1 et 2 et partez du cluster k3d. **Ne sacrifiez jamais** les étapes 9 et 10 : les garde-fous et l'incident sont le cœur du message.

Grille d'évaluation

Critère	Poids	Ce qu'on regarde
La chaîne fonctionne de bout en bout	25 %	Un changement atteint le cluster
Les garde-fous sont actifs et testés	25 %	Un blocage réel est démontré
Qualité des artefacts produits	20 %	laC lisible, manifestes conformes
Diagnostic et post-mortem	15 %	Cause étayée, actions assignées
Capitalisation	15 %	Plugin valide et réutilisable

Le message central de la formation

Un binôme qui livre une chaîne **partielle mais correctement cadrée** passe devant un binôme qui livre tout **sans garde-fou**.

Le vrai livrable : votre plan 30 / 60 / 90 jours

Horizon	Attendu	Exemple
30 jours	Un CLAUDE.md et deux skills sur un dépôt réel ; les trois hooks de garde.	« Le dépôt laC a ses conventions et son anti-secret. »
60 jours	Le plugin d'équipe publié sur un marketplace interne ; MCP GitHub branché.	« Les 12 ingénieurs ont la même configuration. »
90 jours	L'agent dans la CI en revue d'infra ; KPI mesurés (MTTD, MTTR, délai de revue).	« On sait chiffrer ce que ça nous a apporté. »

Les trois questions à traiter

1. Quel est votre **cas d'usage n°1** — celui qui vous fait perdre le plus de temps aujourd'hui ?
2. Quel **garde-fou** doit être en place **avant** de le lancer sur un dépôt réel ?
3. Qui est **propriétaire** du **CLAUDE.md** et du plugin dans l'équipe ?

Merci.

L'IA propose. L'ingénieur décide et teste.